ВКР: как я хочу её делать — коротко

Студент	Евстигнеев Никита Артёмович, группа P3431, 09.03.01 «Информатика и вычислительная техника»
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.

> **Суть.** ИИ-агент работает на одноплатном компьютере, к которому подключены датчики, камера, исполнительные устройства или мобильный робот. «Думает» внешняя языковая модель (GPT mini/nano через RouterAI), а сведения об оборудовании агент получает через MCP-серверы. Агент ведёт два контекста — пользователя и оборудования — и сам решает, когда и с какой справкой обращаться к модели.
>
> **Главный критерий** (как предложено на встрече) — не надёжность, а **работа системы во времени, её производительность и экономность**. За рабочий день измеряются вызовы модели, токены, затраты в рублях, трафик, память и загрузка процессора платы, задержка реакции; отдельно проверяется работа при предельно жёстких ограничениях ресурсов.

Почему мой подход хорош именно для одноплатника

Что	Как устроено	Что уже видно (предварительно)
Лёгкий агент	Один статический исполняемый файл на Go без зависимостей: не нужны ни Node.js, ни Python, ни контейнеры	в простое около 13 МиБ памяти в виртуальной плате; около 26–29 МиБ при проигрывании рабочего дня (на ноутбуке, RSS). Статическая сборка: по требованиям Go должна запускаться и на старой системе Jetson Nano (Ubuntu 18.04, ядро 4.9), где OpenClaw и NanoClaw штатно не работают; на самой плате пока не проверялась
Модель вызывается, только когда нужно	Событийный фильтр и локальные правила на плате вместо «проверки по таймеру» раз в 30 минут	ожидается меньше вызовов, токенов и пробуждений процессора; прямого замера «таймер против фильтра» ещё нет (первые ускоренные прогоны испортила ошибка стыка агента и эмулятора помещения); в пробном сравнении с аналогами — 77 вызовов против 90–102
Короткий контекст	Стабильное начало запроса в расчёте на кэш провайдера, компактный каталог инструментов, состояние оборудования по требованию, сокращение истории	около 2,7 тыс. входных токенов на один вызов модели при ответе на реплику против 8,1–8,3 тыс. у PicoClaw (профиль с seahorse) и 10,8–11,5 тыс. у nanobot; по сумме входных токенов за фрагмент дня против самой экономной найденной настройки PicoClaw — примерно в 1,9 раза меньше

Робот: сводки вместо сырых данных	Свой MCP-адаптер робота: 5 инструментов, сводки и события «доехал / прервано / препятствие», кадр только по запросу	в 3,7–10 раз меньше входных токенов на задачу и примерно в 110–220 раз меньше трафика на поездку, чем у готового ros-mcp-server; память 11–15 МиБ против ≈110 МиБ
Камера на плате (планируется)	Детектор объектов будет работать на плате, в модель уйдёт текстовая сводка, а не кадр; сервер камеры ещё не написан	по токенам кадр дороже сводки в несколько раз, а у некоторых моделей — в сотни раз
Безопасность приводов почти без затрат	Детерминированная проверка каждой команды в MCP-сервере, а не правила в запросе к модели	около 0,5 мкс на проверку
Предельные ограничения	«Лестница ограничений» на виртуальной плате: память до 32 МиБ, процессор до 10 % ядра, плохая сеть, медленная карта памяти	лимиты памяти и процессора проверены (при 32 МиБ процесс, которому нужно 64 МиБ, завершается; квота 10 % даёт 0,3 с процессора за 3 с), средства для сети и диска (tc netem, io.max) доступны; «карта выживаемости» агентов — следующий этап

Все числа — предварительные: прогоны шли на модели-заглушке (без реального API) и при параллельной нагрузке на ноутбук. Итоговые замеры будут сделаны по протоколу на свободной машине, на реальной модели и на плате. Сравнение с аналогами прошло независимое ревью. Оно нашло перекосы в пользу моего агента: неудачную настройку PicoClaw, сжатие времени у nanobot и разную основу сравнения памяти. В перепрогоне ревьюера с более экономной настройкой PicoClaw разрыв по входным токенам сократился с 3,1 до 1,9 раза; стенд сравнения исправляется, после этого числа будут получены заново.

Что я посмотрел

Просмотрел несколько сотен научных работ и проектов; ключевые источники проверены по первоисточнику (список — `research/new_topic/SOURCES.md`). Существующие агенты делятся на несколько классов: личные агенты на собственном оборудовании пользователя (OpenClaw и его облегчённые варианты NanoClaw, PicoClaw, nanobot), умный дом (Home Assistant с языковой моделью и MCP), агенты для роботов на ROS (ROSA, ros-mcp-server), универсальные фреймворки и научные системы. Схема «тонкий агент + внешняя модель + MCP к оборудованию» уже стала распространённой, поэтому новизна не в ней. **Пробел**: эти агенты меряли на коротких задачах. Измерений того, как они ведут себя в течение рабочего дня на одноплатнике и при жёстких ограничениях, в найденных источниках нет. Подробно — в документе «Что я изучил».

Что в работе моё. (1) Измеренный профиль «рабочего дня» агента на одноплатнике — вызовы, токены, рубли, трафик, память, процессор — и условия, при которых каждый приём экономии окупается. (2) Агент, который подписывается на изменения оборудования по MCP: у PicoClaw и nanobot MCP-клиент берёт у сервера только инструменты, события датчиков до них сами не доходят. (3) MCP-сервер оборудования, который сам строит описание датчиков и реле по данным ядра Linux (готовой реализации в проверенных источниках не найдено). (4) Компактный адаптер робота: сводки и события вместо сырых потоков.

Что уже сделано

- Работающий прототип системы: агент на Go, MCP-серверы датчиков и реле, адаптер робота, генератор «рабочего дня» по записям датчиков реальных домов (набор CASAS) и данным исследований использования голосовых ассистентов, эмулятор помещения, модель-заглушка для бесплатных прогонов, метрики и статистика.
- Стенд: виртуальная «плата» (ARM, 4 ядра, 4 ГБ памяти) с регулируемыми ограничениями и виртуальная машина-симулятор робота: ROS 2, Gazebo, TurtleBot3, навигация Nav2. С «платы» робот управляется по сети через мой MCP-адаптер (пока тестовым MCP-клиентом; подключение самого агента — следующий шаг).
- Установлены и подключены к тому же стенду аналоги для сравнения: PicoClaw и nanobot. NanoClaw в замерах не участвует: внутри у него Claude Agent SDK, с GPT mini через RouterAI он штатно не работает, нужны Docker и Node.js ⩾ 22; из него взят приём «скрипт решает, будить ли модель».
- Серверы оборудования и робота, стенд «рабочего дня», сэмплер и обвязка аналогов прошли независимое ревью с замерами; ревью самого агента было прервано и будет повторено. Найденные ошибки исправляются. Подробно — в документе «Что уже сделано и первые замеры».

Что прошу и что хочу согласовать

1. **Доступ к RouterAI** (командный аккаунт с лимитом): по расчёту основные эксперименты стоят около 1,3–1,5 тыс. ₽ при gpt-4o-mini, с запасом ×1,5 — около 1,9–2,3 тыс. ₽ (если основной моделью станет gpt-5-mini — около 3,8 тыс. ₽).
2. **Доступ к Jetson Nano** на кафедре и уточнение модификации платы (4 ГБ или 2 ГБ) — для переноса с виртуальной платы и реальных 8-часовых прогонов.
3. При возможности — **периферия**: USB-камера, датчик температуры, датчик движения, реле.
4. **Согласовать** два сценария (помещение с датчиками и мобильный робот) и формулировку цели (как в плане ВКР): сократить суточные затраты на обращения к внешней модели (вызовы, входные токены, рубли) относительно периодической фоновой проверки — штатного режима аналогов, а относительно PicoClaw в лучшей штатной настройке — не превысить их; при этом полнота реакции на значимые события не хуже (допуск 5 п. п.), небезопасных команд — ноль.

Что я изучил: существующие ИИ-агенты для граничных устройств

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.; данные о проектах (звёзды на GitHub, версии) — на 6 октября 2026 г.

> **Итог обзора в трёх пунктах.** (1) Схема «тонкий агент на устройстве + внешняя языковая модель + MCP к оборудованию» уже стала распространённой практикой, поэтому новизна работы не в самой схеме. (2) Существующие агенты оценивались лишь на коротких задачах; многочасового профиля «вызовы модели + токены + рубли + память + процессор» на одноплатнике, а тем более при жёстких ограничениях ресурсов, в проверенных источниках нет. (3) Для робота и камеры главная статья затрат — сырые данные датчиков: их нельзя отдавать модели напрямую.

1. Классы существующих агентов

Класс	Примеры	Где языковая модель	Что важно для ВКР
Личные агенты на собственном оборудовании пользователя и их облегчённые варианты	OpenClaw; NanoClaw, PicoClaw, nanobot, ZeroClaw; агенты на микроконтроллерах ESP32	облако	ближайшие аналоги и основа сравнения
Умный дом и голосовые ассистенты	Home Assistant с языковой моделью и MCP-сервером; xiaozhi-esp32	облако или локально	массовая практика «оборудование → MCP → модель», часто на Raspberry Pi
Робототехника (ROS)	ROSA (NASA JPL), ros-mcp-server, Nav2-MCP; агентные «ОС» роботов	облако или машина оператора	сценарий с роботом; пока в основном исследования, а не массовое использование
Экосистема NVIDIA Jetson	jetson-containers, руководства по запуску OpenClaw на Jetson Orin, jetson-inference	на борту (Orin)	для старого Jetson Nano применимо мало: локальные модели на нём медленные
Универсальные фреймворки	LangChain/LangGraph, OpenAI Agents SDK, smolagents и др.	любая	тяжёлые для слабых плат; в лёгких агентах вместо них реализован собственный минимальный цикл
Промышленный IoT	AWS Greengrass + облачный агент, решения Microsoft и Siemens	облако или локальный сервер	схема «дешёвый локальный фильтр → облачный агент»

Научные системы	SayCan, Code as Policies, ChatGPT for Robotics, Sasha, SimuHome, IoT-MCP	в основном облако	методы оценки, метрики, обоснование

2. Ближайшие аналоги: что они собой представляют и сколько потребляют

Система	Язык и запуск	Память (сторонние замеры; где указано — наш)	Старый Jetson Nano (Ubuntu 18.04)
OpenClaw (≈391 тыс. звёзд)	TypeScript, Node.js ⩾ 24.16	медиана пиковой памяти ≈2,9 ГиБ (Docker на рабочей станции) [1]	штатно не запускается: сборки Node.js требуют более новой системы
NanoClaw (≈31 тыс.)	TypeScript, Node.js ⩾ 22, Docker	—	штатно не запускается
PicoClaw (≈30 тыс.)	Go, один исполняемый файл	≈31 МБ пиковой памяти на Raspberry Pi 4 [2]; в нашем пробном прогоне (v0.3.1, ноутбук) — ≈42 МиБ	должен запускаться (статическая сборка)
nanobot (≈49 тыс.)	Python	≈130 МБ на Raspberry Pi 4 [2]; ≈136 МиБ (версия 0.1.5) [1]; в нашем пробном прогоне (версия 0.3.5) — ≈166 МиБ	только с ручной сборкой одной зависимости
ZeroClaw	Rust	≈20 МБ [2]	сборка musl статическая — должна запускаться (сборка gnu — нет)
ros-mcp-server	Python, MCP-мост к роботу через rosbridge	≈110 МиБ (собственный замер)	—

Какие приёмы экономии уже есть у аналогов: фильтр «будить ли модель» (NanoClaw), правила, которые модель создаёт один раз и которые затем исполняются без неё, событийные триггеры вместо опроса, облегчённый контекст фоновой проверки, лимиты вызовов, отложенная загрузка описаний инструментов. Эти приёмы в работе не выдаются за новые; измеряется, когда и насколько каждый из них окупается. Чего у PicoClaw и nanobot нет (проверено по их коду): их MCP-клиенты берут у сервера только инструменты и не подписываются на изменения ресурсов, поэтому события датчиков до них сами не доходят — на стенде их доставляет внешний фильтр.

3. Почему языковая модель внешняя (как и решили на встрече)

- На одноплатнике с несколькими гигабайтами памяти практически пригодны модели примерно до 1,5 млрд параметров, но и они медленные: на Raspberry Pi 5 — 5–15 токенов в секунду [3]; на оригинальном Jetson Nano модели около 1 млрд параметров дают 4–7 токенов в секунду (любительские замеры сообщества).
- В независимом замере на Raspberry Pi 4 ни один из агентов не справился с задачами на локальных малых моделях (Qwen2.5-1.5B, SmolLM2-360M) [2].
- В реальных умных домах облачные модели выбирают заметно чаще локальных: по данным открытой аналитики Home Assistant, интеграцию Gemini используют около

34,8 тыс. установок, OpenAI — 24,5 тыс., локальную Ollama — 12,6 тыс.

4. Протокол MCP и «контекст оборудования»

MCP (Model Context Protocol) — открытый протокол, по которому агент получает от серверов **инструменты** (вызвать функцию: прочитать датчик, включить реле, поехать к точке) и **ресурсы** (данные, на изменения которых можно подписаться). В 2025–2026 гг. он стал стандартом де-факто для доступа агентов к оборудованию: MCP-сервер Home Assistant (около 25,8 тыс. установок), ros-mcp-server для роботов, xiaozhi-esp32 для микроконтроллеров [4, 5]. Практика подтверждает топологию, выбранную в работе: на стороне оборудования работает MCP-сервер, агент выступает клиентом.

5. Что я взял из научных работ

- **Определения** граничного ИИ и агента [6, 7]; многоуровневая архитектура, в которой нижние уровни работают без «рассуждающего» модуля [8].
- **Оценка агентов по затратам**, а не только по точности, с обязательным простым базовым вариантом [9].
- **Реалистичная нагрузка дня**: частота команд голосовым ассистентам из многомесячного исследования 88 домов (в среднем 2,6; 5,6 и 30 команд в сутки у разных групп пользователей) [10] и записи датчиков реальных домов CASAS [11].
- **Сокращение истории**: простое скрытие старых результатов инструментов не хуже составления сводки моделью и дешевле [12].
- **Безопасность приводов**: правила, записанные в запросе, модель соблюдает не полностью, поэтому нужен отдельный детерминированный модуль проверки [13].
- **MCP для датчиков** на граничных узлах: среднее время ответа 205 мс, 12-часовой прогон, но без учёта токенов и стоимости [5].
- **Бенчмарки умного дома** для формулировки задач и проверки по итоговому состоянию устройств [14, 15].

6. Пробел, который закрывает работа

Существующие решения либо ориентированы на функциональность и считают в лучшем случае токены и стоимость отдельных запросов (например, /usage cost в OpenClaw), но не профиль затрат и ресурсов платы за многочасовую работу (OpenClaw, PicoClaw, nanobot), либо дают доступ к оборудованию без агента и без метрик (ros-mcp-server, MCP-сервер Home Assistant). Независимые измерения агентов этого класса на одноплатниках ограничены отдельными задачами и не охватывают ни многочасовую работу, ни предельно слабое оборудование. Поэтому в работе разрабатывается агент с MCP-доступом к оборудованию и встроенным учётом ресурсов и затрат, а также методика оценки его работы во времени, включая сравнение с аналогами на одной и той же нагрузке.

Источники (основные)

1. Salman A. Resource Constraints and Performance in Agentic AI Systems / A. Salman, M. Halgamuge, T. Susnjak. – 2026. – arXiv:2608.27886 (препринт).

2. Dehigaspitiyage Don S. Y. Benchmarking Lightweight AI-Agent Frameworks on a 2 GB Raspberry Pi 4: Framework Overhead, Offline Feasibility, and Fleet Density. – Zenodo, 2026. – DOI 10.5281/zenodo.20593332 (препринт).
3. Nguyen T. An Evaluation of LLMs Inference on Popular Single-board Computers / T. Nguyen, T. Nguyen. – 2025. – arXiv:2511.07425 (препринт).
4. Model Context Protocol : specification, version 2025-11-25. – URL: https://modelcontextprotocol.io/specification/2025-11-25 (дата обращения: 06.10.2026).
5. IoT-MCP: Bridging LLMs and IoT Systems Through Model Context Protocol / N. Yang, G. Lyu, M. Ma [et al.] // Proceedings of WiNTECH '25. – 2025. – P. 73–80. – DOI 10.1145/3737895.3768303.
6. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing / Z. Zhou, X. Chen, E. Li [et al.] // Proceedings of the IEEE. – 2019. – Vol. 107, № 8. – P. 1738–1762. – DOI 10.1109/JPROC.2019.2918951.
7. Wooldridge M. Intelligent agents: theory and practice / M. Wooldridge, N. R. Jennings // The Knowledge Engineering Review. – 1995. – Vol. 10, № 2. – P. 115–152. – DOI 10.1017/S0269888900008122.
8. Brooks R. A. A robust layered control system for a mobile robot // IEEE Journal on Robotics and Automation. – 1986. – Vol. 2, № 1. – P. 14–23. – DOI 10.1109/JRA.1986.1087032.
9. AI Agents That Matter / S. Kapoor, B. Stroebl, Z. S. Siegel [et al.] // Transactions on Machine Learning Research. – 2025. – URL: https://openreview.net/forum?id=Zy4uFzMviZ.
10. Understanding the Long-Term Use of Smart Speaker Assistants / F. Bentley, C. Luvogt, M. Silverman [et al.] // Proceedings of the ACM on IMWUT. – 2018. – Vol. 2, № 3. – Art. 91. – DOI 10.1145/3264901.
11. CASAS: A Smart Home in a Box / D. J. Cook, A. S. Crandall, B. L. Thomas, N. C. Krishnan // Computer. – 2013. – Vol. 46, № 7. – P. 62–69. – DOI 10.1109/MC.2012.328.
12. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management / T. Lindenbauer, I. Slinko, L. Felder [et al.] // DL4C Workshop at NeurIPS 2025. – 2025. – URL: https://openreview.net/forum?id=OHVzruJl5k.
13. Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents / Z. Yang, S. S. Raman, A. Shah, S. Tellex // ICRA 2024. – 2024. – P. 14435–14442. – DOI 10.1109/ICRA57147.2024.10611447.
14. SimuHome: A Temporal- and Environment-Aware Benchmark for Smart Home LLM Agents / G. Seo, J. Yang, J. Pyo [et al.] // ICLR 2026. – 2026. – URL: https://openreview.net/forum?id=LCS1WsGvha.
15. HomeBench: Evaluating LLMs in Smart Homes with Valid and Invalid Instructions Across Single and Multiple Devices / S. Li, Y. Guo, J. Yao [et al.] // Proceedings of ACL 2025 (Volume 1: Long Papers). – 2025. – P. 12230–12250. – DOI 10.18653/v1/2025.acl-long.597.

Полная карта агентов с проверкой фактов — `research/new_topic/landscape/AGENT_LANDSCAPE.md`; полный список проверенных источников — `research/new_topic/SOURCES.md`.

Мой подход: экономный ИИ-агент для одноплатного компьютера

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.

> **Идея.** На слабой плате дорого всё: память, процессорное время, каждое пробуждение, каждый сетевой запрос, а с внешней моделью — ещё и каждый токен. Поэтому агент устроен так, чтобы **как можно реже и как можно короче** обращаться к модели и при этом не пропускать важное: локально — всё, что можно решить быстро и однозначно; в облако — только сжатая справка.

1. Состав системы

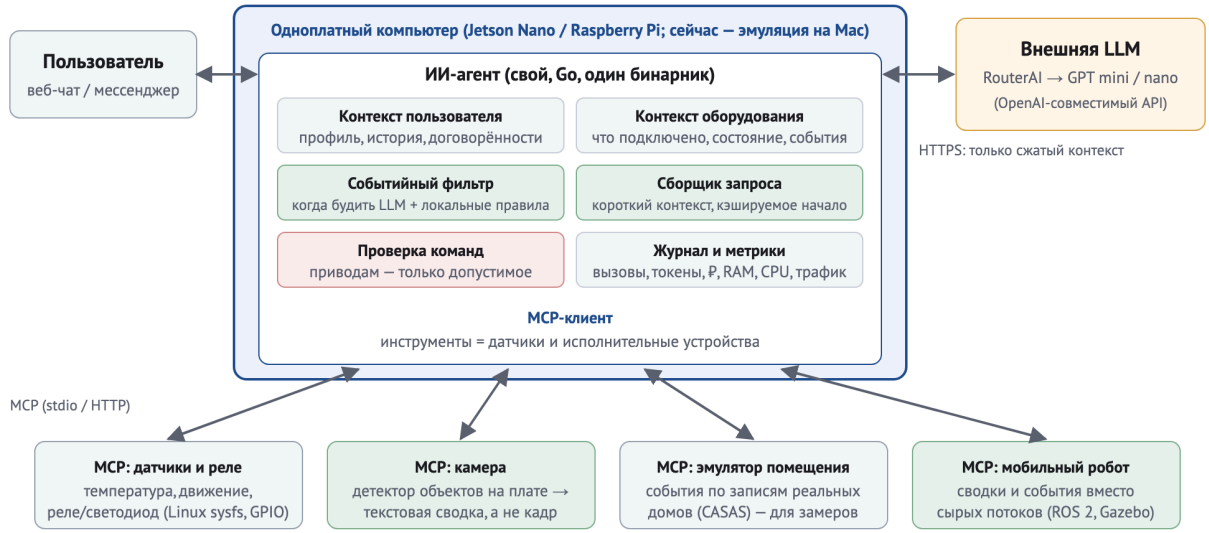


Рисунок 1 — Состав системы. Зелёным отмечены механизмы, которые сокращают вызовы модели и объём передаваемых данных; красным — проверка команд исполнительным устройствам (на схеме она показана в составе агента, а реализована в MCP-сервере оборудования, между моделью и приводами).

- **Агент** (рабочее название EdgeCtx) — собственная программа на Go: один статический исполняемый файл без зависимостей (сейчас около 13,5 МБ). Проверен в виртуальной плате (Ubuntu 24.04, ARM); по требованиям Go должен запускаться и на Raspberry Pi 5, и на устаревшей системе Jetson Nano (Ubuntu 18.04, ядро 4.9), где аналоги на Node.js штатно не запускаются. На реальных платах это ещё предстоит проверить.
- **Внешняя языковая модель** — GPT mini/nano через RouterAI (OpenAI-совместимый интерфейс). Для разработки и ускоренных прогонов — модель-заглушка с «правильными ответами», чтобы бесплатно измерять затраты политик агента.
- **MCP-серверы оборудования**: датчики и реле (описание оборудования строится автоматически из данных ядра Linux); камера с детектором объектов на плате (запланирована, ещё не написана); эмулятор помещения по записям реальных домов; компактный адаптер мобильного робота.

2. Механизмы, которые делают агента экономным

Многие приёмы по отдельности уже встречаются у аналогов (см. «Что я изучил», раздел 2) и за новые не выдаются. Новое — их сочетание в одном агенте для одноплатника, MCP-сервер, который сам строит описание оборудования по данным ядра Linux, и измерение того, при какой нагрузке каждый приём окупается.

Механизм	Что делает	Что экономит
Событийный фильтр и локальные правила	Решает, будить ли модель: пороги, подавление дребезга, объединение событий, «конец эпизода». Правила, которые модель один раз создала («если дверь открыта после 22:00 — напомни»), дальше исполняются на плате без модели	вызовы модели, токены, пробуждения процессора
Два контекста	Контекст пользователя: факты-предпочтения, история с политиками сокращения (окно, скрытие старых результатов инструментов, сводка). Контекст оборудования: перечень устройств — кратко в постоянной части запроса, состояние — снимком, изменениями или по требованию	входные токены, рубли
Стабильное начало запроса	Неизменная часть (инструкция, описания инструментов, перечень устройств) всегда идёт первой и сериализуется байт в байт одинаково; всё изменчивое — в конце	стоимость за счёт кэша провайдера (кэшированные токены в 2–10 раз дешевле; работает ли кэш через RouterAI — проверяется первым же ключом)
Компактный каталог инструментов	Несколько общих инструментов («список устройств», «прочитать свойство», «записать свойство») вместо отдельного инструмента на каждое свойство	входные токены в каждом запросе
Сводки вместо сырых данных	Камера: детектор на плате → «в комнате 1 человек». Робот: «ближайшее препятствие 0,6 м спереди» вместо полного скана лидара (около 4,4 тыс. токенов)	токены, трафик, время ответа
Асинхронные долгие действия	«Поехать к двери» возвращает управление сразу, а результат приходит событием «доехал / прервано / препятствие» — без опроса и без потока промежуточных статусов	время хода агента (он не блокируется на 16–90 с) и трафик (без потока промежуточных статусов); цена — один дополнительный вызов модели на отчёт о прибытии, если пользователь его ждёт

Детерминированная проверка команд	Каждая команда приводу проходит проверку в MCP-сервере: наличие в реестре, диапазон, свежесть состояния, блокировки, частота, подтверждение необратимых действий; при потере связи — безопасное состояние	безопасность без лишних вызовов модели; около 0,5 мкс на проверку
Учёт собственных ресурсов	Агент ведёт суточный бюджет (вызовы и входные токены): после 80 % лимита отклоняет фоновые задачи, при исчерпании обслуживает только пользователя. Состояние платы (память, процессор, температуру) модель может запросить инструментом board_health сервера оборудования; автоматического урезания по нагрузке платы пока нет	предсказуемый расход за день
Лёгкая реализация	Статический исполняемый файл на Go, одно постоянное соединение с моделью, без потоковой передачи ответов, без лишних потоков и таймеров; все таймеры работают от общих часов (это позволяет проигрывать день ускоренно)	память, пробуждения, энергия

3. Два сценария

- **«Умное помещение / лаборатория».** Датчики температуры, движения, открытия двери и освещённости; реле (свет, вентилятор); камера. Рабочий день собирается из реальных данных: частота команд пользователя — по многомесячному исследованию голосовых ассистентов, события датчиков — по записям реальных домов CASAS. В сценарий дня входят значимые ситуации («дверь открыта ночью», «перегрев»), которые агент должен заметить сам, и вопросы о прошлом («когда в последний раз открывали дверь?»).
- **«Мобильный робот».** TurtleBot3 с камерой и лидаром в симуляторе Gazebo под ROS 2, навигация Nav2. Обход точек по расписанию (локально, без модели на каждом шаге), команды пользователя («подъезжай к двери», «что в коридоре?»), события («препятствие», «человек в кадре», «низкий заряд»). Симулятор работает в отдельной виртуальной машине, агент — на «плате», связь по сети, как при реальном развёртывании.

4. Как проверяется эффективность

- **Рабочий день.** Один и тот же день проигрывается на разработанном агенте, на его вариантах с отключёнными механизмами (чтобы оценить вклад каждого), на агенте «только с периодической фоновой проверкой» (так штатно работают аналоги) и на PicoClaw в лучшей штатной настройке; nanobot в штатной настройке сравнивается по фоновому потреблению и памяти, а также в ускоренных прогонах. Для робота — сравнение своего адаптера с готовым ros-mcp-server на одном и том же агенте.

- **Метрики за день:** вызовы модели; входные, выходные и кэшированные токены; рубли; трафик; задержка «событие → действие»; память и процессор платы во времени (в том числе рост памяти за 8 часов); энергопотребление (при наличии датчика мощности); полнота реакции на значимые события; число небезопасных команд (должно быть 0).
- **Ускоренный режим.** У агента виртуальные часы, поэтому 16-часовой день проигрывается за минуты; для аналогов, у которых виртуальных часов нет, время дня сжимается в *k* раз с соответствующим пересчётом таймеров. Для каждой метрики указано, корректна ли она при сжатии (вызовы и токены — да, если все периоды аналога пересчитываются точно; процессор в сутки — нет). Например, при сжатии в 60 раз у nanobot выключилось фоновое сжатие истории, поэтому для него берётся сжатие в 15 раз. Итоговые цифры будут подтверждены реальными 8-часовыми прогонами на плате.
- **Лестница ограничений.** Тот же день — при всё более жёстких лимитах виртуальной платы: память 1024 → 512 → 256 → 128 → 64 → 32 МиБ, процессор 4 ядра → 1 ядро → 50 → 25 → 10 % ядра, сеть от локальной до 3G/EDGE с потерями и обрывами, медленная карта памяти. Результат — «карта выживаемости»: где какой агент ещё работает и как растут задержка и затраты. Ядро ноутбука в несколько раз быстрее ядра Jetson Nano, поэтому ступени по процессору калибруются по плате.
- **Статистика:** парные доверительные интервалы по трассам дня, проверка «не хуже» с допуском, пороги фиксируются до прогонов.

5. Почему это хорошо работает на одноплатнике

1. Плата делает только то, что у неё получается быстро и дёшево (фильтрация событий, правила, проверка команд, распознавание объектов), а дорогое и медленное — «рассуждение» — вызывается редко и с короткой справкой.
2. Сам агент почти не потребляет ресурсов: в простое около 13 МиБ памяти и единичные пробуждения в минуту, поэтому плата остаётся свободной для датчиков, камеры и детектора.
3. Расход на модель предсказуем и ограничен бюджетом; агент и так передаёт мало данных, поэтому плохая сеть сказывается на нём меньше, а локальные правила продолжают работать и без связи.
4. Один и тот же исполняемый файл переносится между платами без установки зависимостей — это будет проверено на старом Jetson Nano и в виртуальной плате с жёсткими лимитами (серверу оборудования для GPIO на ядре 4.9 ещё нужна поддержка старого интерфейса ядра).

Первые замеры приведены в документе «Что уже сделано и первые замеры»; по событийному фильтру и камере пока есть только расчётные ориентиры (план ВКР, гипотезы H1 и H5). Итоговые результаты будут получены по протоколу экспериментов.

Что уже сделано и первые замеры

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.

> **Важная оговорка.** Все числа ниже — **предварительные ориентиры**. Прогоны шли на модели-заглушке (без реального API), в виртуальных машинах и при параллельной нагрузке на ноутбук. Они показывают, что стенд работает и в какую сторону идут эффекты. Итоговые результаты будут получены по протоколу: на свободной машине, с повторами, на реальной модели и на плате.

1. Программная часть

Компонент	Назначение	Язык	Код / тесты, строк
Агент EdgeCtx	часы (реальные и виртуальные), событийный фильтр и правила, планировщик с бюджетом, контексты пользователя и оборудования, сборка запроса, клиент модели, MCP-клиент, журнал трасс	Go	7 927 / 3 643
MCP-сервер оборудования	датчики и GPIO из данных ядра Linux, два варианта каталога инструментов (общие инструменты или отдельный инструмент на каждое свойство), проверка команд приводам, аренда управления с возвратом в безопасное состояние	Go	7 293 / 3 540
MCP-адаптер робота	статус, асинхронная поездка, обход, просмотр (сводка или кадр по запросу), остановка; события вместо опроса	Go	5 501 / 1 625
Монитор ресурсов (sysmon)	память (PSS/RSS), процессор, ввод-вывод, трафик процессов и cgroup во времени	Go	2 835 / 1 271
Стенд «рабочего дня»	генератор дня (CASAS + частоты команд), эмулятор помещения, модель-заглушка, проигрыватель дня, метрики, модель стоимости, статистика, монитор трасс, обвязка аналогов	Python	≈17 000 / ≈8 100
Стенд робота	создание и настройка виртуальной машины, запуск симулятора, замеры нагрузки и трафика	shell, Python	≈3 700 / —

Всего около 62 тыс. строк кода и тестов; сейчас все тесты проходят (Go — 46 пакетов, Python — 394 теста). **Взято готовым**, а не разработано мной: ROS 2, Gazebo, TurtleBot3, Nav2 и rosbridge (симулятор робота); официальные MCP SDK для Go и Python; аналоги PicoClaw, nanobot и ros-mcp-server; записи датчиков CASAS и фразы команд MASSIVE; tiktoken для подсчёта токенов. Серверы оборудования и робота, стенд «рабочего дня», сэмплер, монитор и обвязка аналогов прошли независимое ревью с замерами; ревью

самого агента было прервано и будет повторено. Часть исправлений уже внесена, но ещё не перепроверена; найденные ошибки — в списке ниже.

2. Стенд

- **Виртуальная плата**: Ubuntu 24.04, ARM, 4 ядра, около 4 ГБ памяти. Проверено, что ограничения работают: при лимите 32 МиБ процесс, которому нужно 64 МиБ, завершается системой, как на настоящей плате; квота 10 % ядра даёт ровно 0,3 с процессорного времени за 3 с; доступны эмуляция плохой сети и медленного диска.
- **Симулятор робота**: отдельная виртуальная машина с ROS 2 Jazzy, Gazebo Harmonic, TurtleBot3 (камера, лидар, одометрия) и навигацией Nav2. С «платы» робот управляется по сети через MCP-адаптер (пока тестовым MCP-клиентом; подключение агента — следующий шаг). В ускоренном режиме (примерно в 2,5 раза быстрее реального времени) робот под управлением Nav2 дошёл до всех 15 целей из 15.
- **Аналоги**: PicoClaw 0.3.1 и nanobot 0.3.5 установлены (версии и контрольные суммы зафиксированы) и подключены к той же модели-заглушке и тем же MCP-серверам; реплики и события доходят до них через их штатные каналы.
- **Данные**: записи датчиков реальных домов CASAS (открытый набор, лицензия CC BY 4.0).

3. Потребление ресурсов компонентами

Компонент	Память	Прочее
Агент EdgeCtx	≈13 МиБ в простое (виртуальная плата, PSS); ≈26–29 МиБ при проигрывании рабочего дня (ноутбук, RSS; на плате ещё не мерилось)	исполняемый файл ≈13,5 МБ; единичные пробуждения в минуту в простое
MCP-сервер оборудования	≈9–10 МиБ в простое, ≈15 МиБ под нагрузкой, без утечек	проверка команды приводу ≈0,5 мкс
MCP-адаптер робота	≈11–15 МиБ	запуск ≈3 мс; исполняемый файл ≈9,5 МБ
Монитор ресурсов	≈9 МиБ	≈0,2 % ядра при опросе раз в секунду; ≈0,007 % при опросе раз в 30 с

4. Первое сравнение с аналогами (пробный прогон)

Двухчасовой фрагмент рабочего дня, сжатый в 60 раз, модель-заглушка с «правильными ответами», одинаковый сценарий и эмулятор помещения; у аналогов в части прогонов подключён ещё и сервер оборудования (у разработанного агента — нет), на равной основе сравнивают прогоны «только эмулятор».

Система	Вызовов модели	Входных токенов	Входных токенов на вызов (при ответе на реплику)	Память

Разработанный агент	77	≈0,20 млн	≈2,7 тыс.	≈28 МиБ (без сервера оборудования)
PicoClaw, профиль «best» (seahorse)	91	≈0,62–0,64 млн	≈8,1–8,3 тыс.	≈42 МиБ без сервера оборудования; ≈48–59 МиБ с ним
PicoClaw без seahorse и с выключенными лишними инструментами (перепрогон ревьюера)	102	≈0,39 млн	—	—
nanobot	90	≈0,84–0,90 млн	≈10,8–11,5 тыс.	≈166 МиБ без сервера оборудования; ≈167–188 МиБ с ним

Что показало ревью этого сравнения (честно). Настройка PicoClaw, которую мы считали лучшей, по экономности такой не была: без seahorse и с выключенными лишними встроенными инструментами (это штатные настройки PicoClaw) преимущество разработанного агента по сумме входных токенов — примерно в 1,9 раза, а не в 3,1; у PicoClaw при этом 102 вызова против 77. Это временный перепрогон ревьюера, на стенде его ещё надо повторить. У nanobot сжатие времени в 60 раз выключило фоновое сжатие истории; при сжатии в 15 раз у него 96 вызовов и ≈0,78 млн входных токенов. В первой версии сравнения память и токены сравнивались на разной основе (у аналогов — вместе с сервером оборудования); в таблице выше основа уже одинаковая. Настройка PicoClaw и режим сжатия исправляются в обвязке аналогов до основных прогонов.

5. Робот: свой адаптер против готового ros-mcp-server

Показатель	Свой адаптер	ros-mcp-server
Описание инструментов в каждом запросе	≈274 токена (5 инструментов)	≈2 221 токен (31 инструмент)
Ответ на «где ты?» / «что впереди?»	≈52 токена (одна сводка на оба вопроса)	≈235 токенов (сырая поза) / ≈4 433 токена (сырой скан лидара)
Входные токены на задачу	в 3,7–10 раз меньше у своего адаптера (например, «что впереди?»: 906 против 9 201)	
Трафик на одну поездку (≈3,7 м)	≈4–8 КБ	≈926 КБ
Трафик в простое (2 мин)	0	—
Кадр с камеры по запросу	10 из 10, ≈0,7 с	4 из 10 (новое соединение на каждый вызов: с «ленивой» камерой стенда первый кадр не успевает прийти)

Команда «стоп» (при живой связи)	≈0,5 мс, робот проезжает по инерции 2–4 см	≈2 с, ≈15 см
Память процесса на плате	≈11–15 МиБ	≈110 МиБ
Время запуска	≈3 мс	≈431 мс

В плане ВКР (гипотеза H6) стоит более ранний ориентир — по прототипу адаптера на Python: в 5,6–11 раз меньше входных токенов и в 50–70 раз меньше трафика. Здесь — итоговая версия на Go: каталог у неё длиннее (274 токена против 229), а асинхронная поездка стоит +1 вызов модели, поэтому выигрыш по токенам немного меньше (3,7–10 раз), а по трафику больше (≈110–220 раз). Ориентир в плане будет обновлён.

Главный вывод: сырые данные робота нельзя отдавать модели — одно сообщение лидара стоит около 4,4 тыс. токенов, две секунды подписки — около 109 тыс. Короткие сводки и события сокращают и токены, и трафик на порядки.

6. Найденные ревью ошибки и что с ними

- **Стенд «рабочего дня»**: несовпадение формата ресурсов эмулятора помещения и сервера оборудования, неучтённые служебные заголовки и запросы сводки в заглушке модели, единицы в одной из метрик. Исправлено, добавлен сквозной тест «настоящий агент + эмулятор + заглушка»; исправления ещё не прошли повторное ревью, поэтому полные дни — после него.
- **Сравнение с аналогами**: в цифрах раздела 4 уже учтены самая экономная настройка PicoClaw, меньшее сжатие времени для nanobot и единая основа сравнения памяти; в обвязке аналогов это ещё нужно сделать настройками по умолчанию.
- **Сэмплер и монитор**: в режиме по умолчанию не записывается завершение процесса из-за нехватки памяти (нужно для «лестницы ограничений»); проверка безопасности команд в мониторе даёт ложные срабатывания на настоящем журнале сервера оборудования.
- **Сервер оборудования**: уведомления тормозили опрос датчиков и сторожевой таймер, а таймеры давали лишние пробуждения в простое (тик 100 мс — в 5 раз больше процессорного времени и в 6 раз больше пробуждений). **Исправлено**: уведомления рассылаются асинхронно, таймеры спят до ближайшего срока (в простое без аренд — ноль пробуждений таймеров), при остановке приводы сразу уходят в безопасное состояние; добавлен регрессионный тест на «зависшего» клиента. Осталось: поддержка старого интерфейса GPIO ядра 4.9, нужного для Jetson Nano.
- **Адаптер робота**: сейчас при обрыве связи или остановке адаптера цель навигации не отменяется — робот доезжает до неё сам, а «стоп» отвечает «ok»; если робота сдвинуть не через адаптер, он сообщает устаревшее место и старый скан. Нужно отменять цель (проверенное исправление — отмена всех целей сервисом Nav2, 4,9 мс); это обязательно до любой работы с реальным роботом.

7. Что дальше

1. До конца октября — дымовой тест кафедрального Jetson Nano (запуск агента, PicoClaw, nanobot и детектора, доступ к RouterAI из сети ИТМО) и решение о плате.
2. Исправить найденные ошибки стенда и обеспечить равные условия сравнения; провести повторное ревью.
3. Первые полные ускоренные дни: периодическая проверка и событийный фильтр, разные политики контекста; сравнение с аналогами.
4. Рабочий день робота; сервер камеры с детектором на плате.
5. Лестница ограничений целиком («карта выживаемости»).
6. С ключом RouterAI: проверка на реальной модели (качество ответов, фактический расход токенов и работа кэша).
7. С доступом к Jetson Nano: перенос и реальные 8-часовые прогоны, калибровка виртуальной платы.
8. Параллельно — текст пояснительной записки: глава 1 (обзор и постановка задачи) к 15 января.

План выпускной квалификационной работы

Студент	Евстигнеев Никита Артёмович, группа P3431
Направление	09.03.01 «Информатика и вычислительная техника»
Руководитель	Сергей Вячеславович _____ (фамилия, должность)
Тема (согласована)	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	6 октября 2026 г. (рабочая версия после встречи с руководителем)

> **Кратко.** ИИ-агент работает на одноплатном компьютере, к которому подключены датчики, камера и исполнительные устройства. Агент сам решает, когда и с какой справкой обращаться к внешней языковой модели (GPT mini/nano через RouterAI), часть событий обрабатывает локальными правилами без модели и проверяет каждую команду исполнительным устройствам; модель нужна для разбора запросов пользователя и нештатных ситуаций. Сведения об оборудовании агент получает через MCP-серверы. Агент ведёт два контекста: **пользователя** (что просил, что предпочитает) и **оборудования** (что подключено, что показывают датчики, что произошло).
>
> Каждый вызов модели требует денег, времени и трафика, поэтому главный вопрос работы — как вести эти контексты и когда вызывать модель, чтобы агент **часами** работал дёшево на маломощном оборудовании и не пропускал важное. Результат оценивается так, как предложил руководитель, — по работе системы во времени: за рабочий день подсчитываются вызовы модели, токены, расходы в рублях, трафик, потребление памяти и загрузка процессора платы, задержка реакции.

1. Тема и рабочее уточнение

На титул (без изменений): «Разработка ИИ-агента для приложений граничного искусственного интеллекта».

Уточнение во введении: ИИ-агент для одноплатного компьютера, использующий внешнюю языковую модель и MCP-доступ к датчикам и исполнительным устройствам и экономно ведущий контексты пользователя и оборудования при многочасовой работе.

Это граничный ИИ в широком смысле обзора [1], который не сводит его к «модели на устройстве»: по шкале из [1] рассуждение относится к уровню 1 (совместный вывод «облако — граница»: данные обрабатываются рядом с источником, в облако уходит только сжатый контекст), а распознавание камерой — к уровню 3 (вывод на устройстве). Кроме того, на плате работают событийный фильтр, локальные правила и проверка команд. Агент понимается в «слабом» смысле [2]: он автономен, реагирует на изменения окружения, действует по собственной инициативе и взаимодействует с пользователем.

Почему языковая модель внешняя. На одноплатнике с несколькими гигабайтами памяти практически пригодны только языковые модели примерно до 1,5 млрд параметров, и они медленные: на Raspberry Pi 5 это порядка 5–15 токенов в секунду, а модели на 3 млрд параметров дают 2–5 токенов в секунду [23]. Открытые модели, сопо-

ставимые по качеству с облачными, требуют десятков гигабайт памяти (например, около 43 ГБ для модели на 70 млрд параметров при 4-битном квантовании [24]). Поэтому «думает» облако, а на плате остаются задачи, которые она решает быстро и локально: распознавание объектов в кадре камеры, фильтрация событий, локальные правила, проверка команд. Для главы 1 это решение будет подкреплено одним контрольным замером на самой плате: время ответа локальной модели около 1 млрд параметров на типичный запрос агента.

2. Что строится

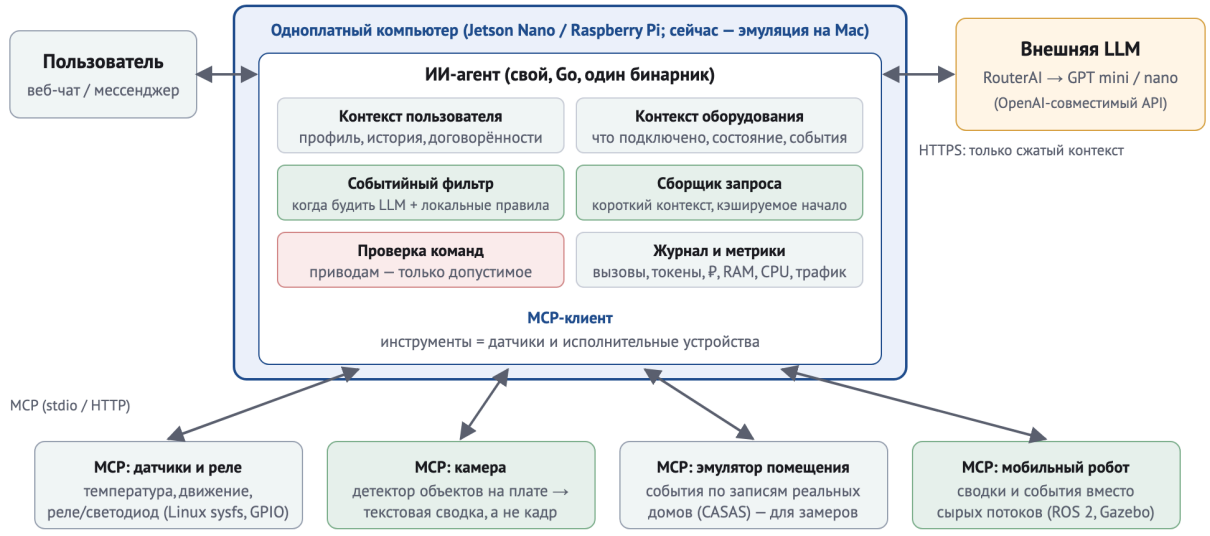


Рисунок 1 — Состав системы. Зелёным отмечены механизмы, которые сокращают число вызовов модели и объём передаваемых данных; красным — проверка команд исполнительным устройствам.

- **Агент** — собственная программа на Go: один статический исполняемый файл без внешних зависимостей. По требованиям Go (ядро Linux ⩾ 3.2) такой файл должен запускаться и на устаревшей системе Jetson Nano (Ubuntu 18.04, ядро 4.9), и на Raspberry Pi; на самой плате это будет проверено дымовым тестом. Внутри — цикл агента, два контекста, событийный фильтр, модуль формирования запроса, проверка команд, журнал метрик. Нижний уровень (локальные правила и проверка команд) работает и без модели — в соответствии с идеей многоуровневой архитектуры управления [3]. Проверка команд вынесена из запроса к модели в отдельный детерминированный модуль, потому что правила, записанные в запросе, модель соблюдает не полностью [10].
- **MCP-серверы оборудования.** Датчики и реле (сведения о подключённом оборудовании берутся из данных, которые уже предоставляет ядро Linux). Камера с детектором объектов на плате: в модель уходит текстовая сводка («в комнате 1 человек»), а не кадр, потому что кадр стоит намного больше токенов: кадр 640×480 — около 360–740 токенов у моделей GPT-5 и GPT-4.1 mini и nano и 2 833–14 167 у gpt-4o-mini против 30–60 токенов у текстовой сводки (от нескольких раз до сотен раз в зависимости от модели). Эмулятор помещения по записям датчиков реальных домов (набор CASAS [5, 25]) нужен для воспроизводимых замеров без реального оборудования. Второй сценарий — **мобильный робот** TurtleBot3 в симуляторе Gazebo под управлением ROS 2 (камера, лидар, одометрия, навигация Nav2).

Симулятор работает в отдельной виртуальной машине, агент — на «плате», связь по сети через rosbridge, как при реальном развёртывании. Для сравнения используется готовый MCP-мост ros-mcp-server, а собственный компактный MCP-адаптер робота отдаёт модели короткие сводки и события вместо сырых потоков датчиков. Обмен идёт по протоколу MCP в ревизии 2025-11-25 [16], которую поддерживают и PicoClaw, и nanobot.

- **Измерительный стенд «рабочего дня».** Один и тот же день — команды пользователя и события датчиков по реальным данным — воспроизводится на разработанном агенте и на аналогах. Потребление ресурсов, вызовы модели и затраты регистрируются как временные ряды.

3. Аналоги и новизна

Система	Что это	Ресурсы
OpenClaw [20]	самый популярный личный ИИ-агент (TypeScript, около 391 тыс. звёзд на GitHub): внешняя языковая модель, инструменты, мессенджеры, периодическая фоновая проверка (heartbeat)	тяжёлый: медиана пиковой памяти около 2,9 ГиБ (2 926,6 МиБ; Docker на рабочей станции, замер на задачу) [11]; на Jetson Nano штатно не запускается: текущая версия требует Node.js ⩾ 24.16, а официальные сборки Node.js для arm64 — glibc ⩾ 2.28 и ядро ⩾ 4.18 (на Nano 2.27 и 4.9)
NanoClaw [19]	облегчённая альтернатива OpenClaw (TypeScript): каждый агент в своём контейнере, внутри — Claude Agent SDK, другие модели подключаются дополнениями. Из него взят приём «скрипт решает, будить ли модель» (`wakeAgent: false`)	Node.js ⩾ 22 и Docker; на Jetson Nano штатно не запускается, с GPT mini через RouterAI штатно не работает — поэтому в замерах не участвует и разбирается в главе 1
PicoClaw [17]	сверхлёгкий агент на Go для недорогих устройств (Sipeed)	заявлено менее 10 МБ (новые сборки 10–20 МБ) [17]; независимый замер — около 31 МБ пиковой памяти на Raspberry Pi 4 [22]
nanobot [21]	лёгкий агент на Python (HKUDS)	около 130 МБ пиковой памяти на Raspberry Pi 4 [22]; медиана 136 МиБ в замере [11] (версия 0.1.5)
Home Assistant + языковая модель + MCP	платформа умного дома с языковой моделью; широко распространена, часто работает на Raspberry Pi	полноценный сервер умного дома
IoT-MCP [6]	научная работа: MCP-серверы для датчиков на граничных узлах	среднее время ответа 205 мс, пиковая память 74 КБ на микроконтроллере, 12-часовой прогон без учёта токенов

Что уже известно. Схема «тонкий агент на плате + внешняя языковая модель + MCP к оборудованию» уже широко распространена, и отдельные приёмы экономии уже

встречаются у этих систем: неизменное начало (префикс) запроса для кэширования на стороне провайдера, фильтры перед вызовом модели, сокращение истории. В работе эти приёмы не выдаются за новые, на их источники даются ссылки. Смежное направление — «операционные системы для агентов» на серверах [15].

Чего нет (пробел). Замеры этих агентов сделаны на коротких задачах. В проверенных источниках не найдено измерений того, как такой агент ведёт себя в течение многочасового рабочего дня на реальном одноплатном компьютере: сколько вызовов модели он делает, сколько токенов, рублей, трафика и памяти расходует и при какой нагрузке каждый приём экономии действительно окупается.

Личный вклад студента:

- собственный агент с двумя контекстами и событийным фильтром;
- MCP-сервер оборудования, который сам строит описание подключённых датчиков и реле по данным ядра Linux, без ручной настройки (готовой реализации этого приёма в проверенных источниках не найдено);
- MCP-сервер камеры с распознаванием объектов на плате;
- стенд «рабочего дня» на реальных данных;
- сравнение с аналогами в одинаковых условиях (одна модель, одни MCP-серверы, один сценарий дня) и в их лучших штатных настройках;
- оценка вклада каждого механизма и условий, при которых он окупается.

4. Цель и задачи

Объект: процесс функционирования ИИ-агента на одноплатном компьютере, использующего внешнюю языковую модель и MCP-доступ к оборудованию.
Предмет: механизмы ведения контекстов пользователя и оборудования и выбора момента вызова модели, их влияние на затраты и потребление ресурсов системы при многочасовой работе.

Цель: сократить суточные затраты на обращения к внешней языковой модели (число вызовов, входные токены, рубли в сутки) ИИ-агента на одноплатном компьютере, чтобы его многочасовая ежедневная работа укладывалась в ограниченный бюджет на модель и ресурсы платы. Сокращение показывается относительно периодической фоновой проверки — штатного режима PicoClaw и nanobot; относительно PicoClaw в лучшей штатной настройке затраты должны быть не выше. Условия: полнота реакции на значимые события не хуже (допуск 5 процентных пунктов), небезопасных команд исполнительным устройствам — ноль. Достижение цели проверяется по 95-процентным доверительным интервалам отношений затрат на рабочем дне на плате.

Задачи:

1. Проанализировать существующих ИИ-агентов для граничных устройств, протокол MCP и подходы к ведению контекста; выявить пробел в существующих исследованиях.
2. Разработать методику оценки «рабочего дня»: сценарий нагрузки по реальным данным, метрики, правила сравнения, методы статистической обработки.
3. Спроектировать архитектуру агента и MCP-серверов оборудования.

4. Реализовать агента, MCP-серверы (датчики и реле, камера с распознаванием, эмулятор помещения, компактный адаптер мобильного робота) и измерительный стенд.
5. Выбрать основную модель среди GPT mini/nano разных версий по стоимости успешного действия на командах стенда; провести сравнительные эксперименты с аналогами в эмуляции и на одноплатном компьютере.
6. Проанализировать результаты: оценить вклад механизмов, определить условия их окупаемости, сформулировать рекомендации по настройке.

Структура записки (40–60 страниц, три главы): глава 1 — обзор, аналоги, почему модель внешняя, пробел (задача 1); глава 2 — методика «рабочего дня», архитектура агента и MCP-серверов (задачи 2–3); глава 3 — реализация, эксперименты, сравнение с аналогами и рекомендации (задачи 4–6).

5. Гипотезы

Пороговые значения фиксируются заранее, до прогонов. Отрицательный исход — тоже результат: он включается в отчёт, а не скрывается.

№	Гипотеза	Как проверяется
H1	**Когда вызывать модель.** Событийный фильтр с локальными правилами требует меньше вызовов модели в сутки, чем периодическая фоновая проверка (так штатно работают PicoClaw и nanobot), при не худшей полноте реакции. Ориентир по расчёту против проверки раз в 30 минут — примерно в 1,8 раза меньше вызовов при 10 командах в сутки и в 2,35 раза при 5,6 команды в сутки (средний пользователь); против лучших настроек PicoClaw исход заранее неизвестен. Выигрыш уменьшается с ростом активности (по тому же расчёту при 30 командах в сутки — около 1,3 раза), поэтому отдельно определяется точка безубыточности: при какой частоте команд и событий фильтр перестаёт выигрывать	рабочий день на записях домов CASAS, не использованных при настройке фильтра; доверительные интервалы отношения вызовов
H2	**Контекст оборудования.** Несколько универсальных инструментов с запросом состояния по необходимости дают меньше входных токенов, чем отдельный инструмент на каждое свойство устройства и снимок оборудования в каждом запросе, без снижения доли успешно выполненных команд. Ориентир статического подсчёта каталога — в 1,4 раза для одной комнаты (10 свойств) и в 3,4 раза для 20 комнат (200 свойств)	набор команд к устройствам комнаты по образцу бенчмарков умного дома [7, 8]; сравнение представлений контекста
H3	**Контекст пользователя.** Скрытие старых результатов вызовов инструментов в сочетании со скользящим окном истории не уступает полной истории по точности ответов и дешевле её, причём выигрыш растёт с активностью пользователя (по расчёту при 10 запросах в сутки стратегии почти равны, при 200 — разница около 4,7 раза)	контрольные вопросы по истории на реальной модели; приём скрытия взят из [9]

H4	**Ресурсы платы.** Рост памяти агента за 8 часов работы не превышает 1 МиБ/ч (проверка эквивалентности, а не «тренд незначим»), а сама память не больше, чем у PicoClaw на той же плате; дополнительно — какие из систем (наш агент, PicoClaw, nanobot) выдерживают 8 часов работы при ограничении памяти 256 МиБ (агент и серверы датчиков, без детектора камеры)	замер потребления памяти и загрузки процессора на плате в течение прогона
H5	**Камера.** Текстовая сводка детектора объектов, работающего на плате, вместо кадра сокращает входные токены и трафик в десятки–сотни раз; оценивается, на какой доле вопросов к сцене сводка уступает кадру по точности и при какой частоте вопросов выгоднее «кадр по запросу»	около 150 вопросов к сцене на плате с USB-камерой в трёх режимах: кадр всегда, сводка, кадр по запросу; сцены без людей (кадры уходят зарубежному провайдеру модели)
H6	**Робот: сводки вместо сырых данных.** Компактный адаптер робота (несколько инструментов, сводки состояния, события «доехал / прервано / препятствие», кадр только по запросу) требует в разы меньше входных токенов и трафика на команду, чем готовый ros-mcp-server, при той же доле выполненных заданий. Ориентир пробных замеров с идеальной моделью: в 5,6–11 раз меньше входных токенов на команду и в 50–70 раз меньше трафика на поездку	рабочий день робота (обход по расписанию, команды пользователя, события) в ускоренном режиме симулятора; сравнение двух адаптеров на одном агенте
H7	**Работа на предельно слабом оборудовании.** Разработанный агент сохраняет работоспособность и экономность при более жёстких ограничениях памяти, процессора и сети, чем аналоги: строится «карта выживаемости» — при каких лимитах (память 1024→32 МиБ, процессор 4 ядра → 10 % ядра, сеть от локальной до EDGE с потерями, медленная карта памяти) каждый агент ещё выполняет рабочий день и как растут задержка и затраты	лестница ограничений в виртуальной плате (cgroup v2, tc netem, io.max) для разработанного агента, PicoClaw и nanobot; калибровка одной-двух ступеней на реальной плате

6. Как измеряется «работа во времени»

Главный критерий работы — **производительность и экономность системы во времени**: сколько вызовов модели, токенов, рублей, трафика, памяти и процессорного времени тратит агент за рабочий день и как быстро он реагирует. Надёжность в цель не входит; безопасность команд исполнительным устройствам — только обязательное ограничение.

- **Сценарий дня.** Команды пользователя задаются по данным исследований использования умных колонок: в среднем 2,6; 5,6 и 30 команд в сутки в кластерах лёгких, средних пользователей и «суперпользователей» (88 домов; медиана по всем домам — 4,1 команды в сутки) [4]. События датчиков — по записям реальных домов CASAS [5, 25].
- **Два режима прогона:**
  - **ускоренный и бесплатный** — на ноутбуке, с виртуальными часами и записанными ответами модели; нужен для сравнения многих вариантов;

    - **реальный** — 8 часов на плате с настоящей моделью (окно рабочего дня 14:00–22:00 по сценарию; «суточные» величины в этом режиме относятся к этому окну, в ускоренном режиме — к активному окну 07:00–23:00).
- **Метрики за сутки:** число вызовов модели; входные и выходные токены; расходы в рублях (по тарифам, зафиксированным на дату экспериментов); трафик; задержка «событие → действие» (95-й процентиль); потребление памяти и загрузка процессора платы во времени; энергопотребление (если есть датчик мощности); полнота реакции на значимые события; число небезопасных команд (должно быть 0).
- **С чем сравнивается:** агент «только с периодической проверкой» — штатная политика PicoClaw и nanobot, реализованная на нашем коде (простой базовый вариант обязателен при совместной оценке агентов по качеству и стоимости [12]); PicoClaw в лучшей штатной настройке — на полном рабочем дне; nanobot — по фоновому потреблению и памяти; варианты разработанного агента с отключёнными механизмами (для оценки вклада каждого). У всех систем одна модель, одни MCP-серверы и один сценарий дня.
- **Статистика:** парные бутстреп-доверительные интервалы по трассам дня, проверка не меньшей эффективности («не хуже») с допуском, пороги фиксируются до прогонов [13].

7. Оборудование и стенд

- **Сейчас (MacBook):**
    - виртуальная машина Linux с ограничением числа ядер и объёма памяти до уровня одноплатного компьютера;
    - эмулятор помещения вместо датчиков;
    - модель-заглушка для бесплатной отладки (0 ₽);
    - вторая виртуальная машина — симулятор робота (ROS 2 Jazzy, Gazebo Harmonic, TurtleBot3, Nav2), рабочий день робота проигрывается в ускоренном режиме (примерно в 2,5 раза быстрее реального времени);
    - «лестница ограничений» той же виртуальной платы: память до 32 МиБ, процессор до 10 % одного ядра, эмуляция плохой сети и медленного диска.

  Реальная модель — после получения ключа RouterAI.
- **Плата:** кафедральный NVIDIA Jetson Nano. Нужно уточнить модификацию: выпускались версии с 4 ГБ и 2 ГБ памяти, версии с 1 ГБ не существует. Если на кафедре окажется более новый Jetson Orin Nano (4 или 8 ГБ, современная система), описанные ниже ограничения старой системы снимаются.
    - У версии на 4 ГБ есть встроенный датчик мощности (INA3221; у версии на 2 ГБ его нет). Он занижает мощность до 50 %, поэтому энергию за день по нему можно считать только после калибровки по внешнему измерителю [14]. Графический процессор для детектора камеры есть у обеих версий.
    - Операционная система на плате устаревшая (Ubuntu 18.04). Статические бинарники на Go (наш агент, PicoClaw) по требованиям должны на ней запускаться; на самой плате это ещё не проверено. OpenClaw и NanoClaw штатно не запускаются: им нужен Node.js ⩾ 24.16 и ⩾ 22 соответственно, а официальные

сборки Node.js требуют glibc ⩾ 2.28 и ядро ⩾ 4.18. Python-серверам MCP нужен Python ⩾ 3.10 (на плате он ставится отдельно, например через uv); nanobot 0.3.5 ставится только со сборкой зависимости tiktoken из исходников. Это довод в пользу Go для агента (глава 2). Плата окончательно выбирается по дымовому тесту в октябре: `free -h`, запуск агента, PicoClaw, nanobot и детектора, доступ к routerai.ru из сети ИТМО.
    - Если плата недоступна или её ресурсов недостаточно, альтернатива — Raspberry Pi 5 с 4 ГБ памяти.
- **Периферия:** USB-камера, датчик температуры (DS18B20 или BME280), датчик движения, реле со светодиодом.
- **Бюджет на модель:** по расчёту около 1,3–1,5 тыс. ₽ на основные эксперименты при gpt-4o-mini как основной модели, с запасом ×1,5 — около 1,9–2,3 тыс. ₽ (если основной станет gpt-5-mini — около 3,8 тыс. ₽). RouterAI поддерживает командные аккаунты с ролями и лимитами расходов [18]; по словам руководителя, оплату может взять на себя факультет.

8. План по срокам

Ориентиры взяты из консультации по ГИА (ПИиКТ). На консультации их назвали примерными; окончательные даты будут объявлены позже.

Срок	Что готово
октябрь — ноябрь 2026	закрепление за руководителем (на слайде 15–30 октября); тема в ИСУ / My.ITMO (на слайде до 30 ноября, зависит от открытия сервиса); дымовой тест кафедрального Jetson Nano и решение о плате до конца октября (чтобы при необходимости успеть купить Raspberry Pi 5); заявка на ключ RouterAI; каркас агента, MCP-серверы и эмулятор помещения на ноутбуке
декабрь 2026	стенд «рабочего дня», ускоренные прогоны, первые сравнения механизмов; ключ RouterAI
к 15 января 2027	глава 1 (обзор, аналоги, пробел), цель и задачи
январь — февраль 2027	перенос на плату, подключение датчиков и камеры; реальные 8-часовые прогоны сразу тройками «наш агент / периодическая проверка / PicoClaw» на одних и тех же сценариях дня
начало марта 2027	проверка 1: главы 1–2 (обзор; методика и архитектура)
март — апрель 2027	добор прогонов при большом разбросе, анализ, глава 3, полный текст (проверка 2 — ориентировочно середина апреля, на слайде — до 30 апреля)
к 15 мая 2027	текст готов; 20–30 мая — предзащита; защита ориентировочно 3–20 июня; за 14 дней до защиты проверка на заимствования уже пройдена (отправка на проверку — с 5 мая)

9. Что нужно

- **От руководителя или факультета:**

- доступ к RouterAI (командный аккаунт с лимитом);
- доступ к Jetson Nano и уточнение его модификации;
- при возможности — датчики, камера, реле;
- согласование объёма работы и формулировки цели.

- **Программное обеспечение (небольшие загрузки):**
  - компилятор Go;
  - Python-пакет MCP;
  - образ Linux для эмуляции;
  - исполняемый файл PicoClaw и Python-пакет nanobot для сравнения, MQTT-брокер для доставки им событий;
  - детектор объектов для камеры и его веса.

Ключевые источники

1. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing / Z. Zhou, X. Chen, E. Li [et al.] // Proceedings of the IEEE. – 2019. – Vol. 107, № 8. – P. 1738–1762. – DOI 10.1109/JPROC.2019.2918951.
2. Wooldridge, M. Intelligent agents: theory and practice / M. Wooldridge, N. R. Jennings // The Knowledge Engineering Review. – 1995. – Vol. 10, № 2. – P. 115–152. – DOI 10.1017/S0269888900008122.
3. Brooks, R. A. A robust layered control system for a mobile robot / R. A. Brooks // IEEE Journal on Robotics and Automation. – 1986. – Vol. 2, № 1. – P. 14–23. – DOI 10.1109/JRA.1986.1087032.
4. Understanding the Long-Term Use of Smart Speaker Assistants / F. Bentley, C. Luvogt, M. Silverman [et al.] // Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies. – 2018. – Vol. 2, № 3. – Art. 91. – P. 1–24. – DOI 10.1145/3264901.
5. CASAS: A Smart Home in a Box / D. J. Cook, A. S. Crandall, B. L. Thomas, N. C. Krishnan // Computer. – 2013. – Vol. 46, № 7. – P. 62–69. – DOI 10.1109/MC.2012.328.
6. IoT-MCP: Bridging LLMs and IoT Systems Through Model Context Protocol / N. Yang, G. Lyu, M. Ma [et al.] // Proceedings of the ACM Workshop on Wireless Network Testbeds, Experimental evaluation & Characterization (WiNTECH '25). – 2025. – P. 73–80. – DOI 10.1145/3737895.3768303.
7. SimuHome: A Temporal- and Environment-Aware Benchmark for Smart Home LLM Agents / G. Seo, J. Yang, J. Pyo [et al.]. – Текст : электронный // International Conference on Learning Representations (ICLR 2026). – 2026. – URL: https://openreview.net/forum?id=LCS1WsGvha (дата обращения: 06.10.2026).
8. HomeBench: Evaluating LLMs in Smart Homes with Valid and Invalid Instructions Across Single and Multiple Devices / S. Li, Y. Guo, J. Yao [et al.] // Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). – 2025. – P. 12230–12250. – DOI 10.18653/v1/2025.acl-long.597.
9. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management / T. Lindenbauer, I. Slinko, L. Felder [et al.]. – Текст : электронный // 4th Deep Learning for Code Workshop (DL4C) at NeurIPS 2025. – 2025. – URL: https://openreview.net/forum?id=OHVzruJl5k (дата обращения: 06.10.2026).
10. Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents / Z. Yang, S. S. Raman, A. Shah, S. Tellex // 2024 IEEE International Conference on Robotics and Automation (ICRA). – 2024. – P. 14435–14442. – DOI 10.1109/ICRA57147.2024.10611447.
11. Salman, A. Resource Constraints and Performance in Agentic AI Systems : preprint / A. Salman, M. Halgamuge, T. Susnjak. – 2026. – arXiv:2608.27886. – URL: https://arxiv.org/abs/2608.27886 (дата обращения: 06.10.2026). – Текст : электронный.
12. AI Agents That Matter / S. Kapoor, B. Stroebl, Z. S. Siegel [et al.]. – Текст : электронный // Transactions on Machine Learning Research. – 2025. – URL: https://openreview.net/forum?id=Zy4uFzMviZ (дата

обращения: 06.10.2026).
13. Hoefler, T. Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results / T. Hoefler, R. Belli // SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. – 2015. – Art. 73. – P. 1–12. – DOI 10.1145/2807591.2807644.
14. Accurate Calibration of Power Measurements from Internal Power Sensors on NVIDIA Jetson Devices / N. Shalavi, A. Khoshsirat, M. Stellini [et al.] // 2023 IEEE International Conference on Edge Computing and Communications (EDGE). – 2023. – P. 166–170. – DOI 10.1109/EDGE60047.2023.00034.
15. AIOS: LLM Agent Operating System / K. Mei, X. Zhu, W. Xu [et al.]. – Текст : электронный // Conference on Language Modeling (COLM 2025). – 2025. – URL: https://openreview.net/forum?id=L4HHkCDz2x (дата обращения: 06.10.2026).
16. Model Context Protocol : specification, version 2025-11-25. – URL: https://modelcontextprotocol.io/specification/2025-11-25 (дата обращения: 06.10.2026). – Текст : электронный.
17. PicoClaw : [репозиторий] / Sipeed. – Версия 0.3.1. – 2026. – URL: https://github.com/sipeed/picoclaw (дата обращения: 06.10.2026). – Текст : электронный.
18. RouterAI : единый API к нейросетям : [сайт]. – URL: https://routerai.ru/ (дата обращения: 06.10.2026). – Текст : электронный.
19. NanoClaw : [репозиторий] / NanoCo. – URL: https://github.com/nanocoai/nanoclaw (дата обращения: 06.10.2026). – Текст : электронный.
20. OpenClaw : [репозиторий]. – URL: https://github.com/openclaw/openclaw (дата обращения: 06.10.2026). – Текст : электронный.
21. nanobot : [репозиторий] / Data Intelligence Lab@HKU (HKUDS). – Версия 0.3.5. – 2026. – URL: https://github.com/HKUDS/nanobot (дата обращения: 06.10.2026). – Текст : электронный.
22. Dehigaspitiyage Don, S. Y. Benchmarking Lightweight AI-Agent Frameworks on a 2 GB Raspberry Pi 4: Framework Overhead, Offline Feasibility, and Fleet Density : preprint / S. Y. Dehigaspitiyage Don. – Версия 0.2. – Zenodo, 2026. – DOI 10.5281/zenodo.20593332. – Текст : электронный.
23. Nguyen, T. An Evaluation of LLMs Inference on Popular Single-board Computers : preprint / T. Nguyen, T. Nguyen. – 2025. – arXiv:2511.07425. – URL: https://arxiv.org/abs/2511.07425 (дата обращения: 06.10.2026). – Текст : электронный.
24. Enabling Novel Mission Operations and Interactions with ROSA: The Robot Operating System Agent / R. Royce, M. Kaufmann, J. Becktor [et al.] // 2025 IEEE Aerospace Conference. – 2025. – P. 1–16. – DOI 10.1109/AERO63441.2025.11068426.
25. Cook, D. CASAS Smart Home dataset – free living, motion, door, activity labels : dataset / D. Cook. – Zenodo, 2025. – DOI 10.5281/zenodo.15708568. – Текст : электронный.

Подробная концепция (полный разбор аналогов, методики, бюджета и рисков, около 60 проверенных источников) — `research/new_topic/THESIS_CONCEPT.md`, `SOURCES.md`; карта существующих агентов — `research/new_topic/landscape/AGENT_LANDSCAPE.md`.